Distributed Storage Mini Project

This project is focused on understanding, implementing and comparing performance of a distributed storage system using different schemes for compression and delta encoding. Use the tools and ideas from the lectures and labs to predict the expected behavior and reason about the observed behavior in your system.

You are expected to work in a team to design and implement a distributed storage system using Python 3. Every team member should participate in all stages of the project: specification, system design, implementation, testing and measurements.

Dataset: You are provided a matching dataset in brightspace with 5 files, each with 5 versions to be used in the testing process.

Report: The team should submit a report that briefly describes the system design and answers the Tasks below. The report should be around 10 pages. You can use a IEEE conference double-column style or a standard single column report style. If you exceed the limit, this is not a problem, but please try to avoid very long reports. Quality in your report is more valuable than page count.

Exam: Each team member takes the oral exam individually. You will need to defend your team's design decisions and answer questions about implementation details. You will also need to show the results of the measurement tasks and reason about the expected and observed performance. The report and results will be the basis for the first half of the exam questions.

Groups: We recommend 2 to 3 students, but can allow 4 in some cases. If you are considering doing the mini-project individually, please contact the lecturer.

Delivery Date: January, 5th 2026

(If you need a day or two of extension, please contact the lecturer for approval)

Interface to the Outside World

Your storage system should provide an interface to external clients. It can be either a standard REST API over HTTP that you can test with a tool like Postman (or a simple web application you create), or a ZeroMQ socket API that is called by a native client application. You are encouraged to use the techniques (and code) from the Labs.

There are only two mandatory functions your storage system has to support: **store** and **retrieve** a file. The latter will retrieve the latest version of the file uploaded to the system. You can also support a retrieve function that enables you to download a specified version, but this one is optional.
Pointers:

- Keep the API and system simple to start with
- Consider using Flask for a REST API

- Consider using Protocol Buffers for a ZeroMQ-based interface
- Consider having a lead node that provides the API and coordinates the process
- Consider a mechanism to keep track of a file when multiple versions are being uploaded, e.g., track/connect new versions based on the file name, expect the uploaded version to have the same name and a '*_v1*' '_v2' indication at the end of the name.

Task 1 - Replication

Design & Development

1. Implement a strategy that can generate 3 full replicas of a file and place them in 3 different nodes out of N in your system. Nodes can be implemented as different processes or containers. The lead node sends the file directly to each of the selected nodes. For this (and following tasks) you will need to support the mechanism that detects whether a new upload is a new version of a previously received file.

Analysis

2. Calculate the time to generate all replicas and distribute them if the connection bandwidth is R bps (bits per second, same speed among nodes and between the client to any node). Use the file size B in bytes as an additional variable in your analysis.

3. Calculate the total download time of a file (of B bytes) with each scheme. Download starts when the client sends the request, and ends when the last byte of the file has finished transferring. Consider the same connection bandwidth as in Point 2.

Measurements

4. Measure for cases $N = 6, 12, 24$ focusing on upload time (including the time to generate all replicas and store them correctly) as well as the total download time. Describe and explain the results. Use the files in the dataset directly for these tests as a baseline for performance.
   Metrics:
   - Upload time for a file, from request to successful, reliable storage
   - Download time for a file, from request to download completion

   Measure each file size at least 100 times and report the measured times with their average, median, min and max values.

Task 2: Individual File Compression

Design & Development

1. Expand the system from Task 1 to support compression. For example, after the file is received in the system, the lead node proceeds to compress the file prior to placing it in the 3 different nodes selected. The goal is to reduce the footprint of each uploaded file.

Analysis

2. Calculate the upload time, expanding the analysis from Task 1 in order to introduce the effect of processing (compression) during the upload time. Consider that compression can be carried out at R_{enc} bps and that the resulting file size upon compression is B_{zip} bytes.

3. Calculate the total download time of a compressed file of B_{zip} bytes (when compressed) when the decompression speed is R_{dec} bps. Download starts when the client sends the request, and ends when the last byte of the file has finished transferring. Consider the same connection bandwidth as in Point 2 above.

4. Compare the upload and download time analysis to the results in Task 1. More specifically, compute the potential gain by dividing the upload time for Task 1 by the upload time for Task 2. Repeat the same process for download times. Report your computed gains based on the various system variables.

Measurements

5. Measure the compressed size of the files prior to upload. Measure the compression/decompression times. This will help you use the analysis to predict the gains with respect to the previous section. Report on the predicted gains.
6. Measure for cases $N = 6, 12, 24$ focusing on upload time (including the time to generate all replicas and store them correctly) as well as the total download time. Like before, use the files in the dataset directly for these tests as a baseline for performance. Describe and explain the results.
   Metrics:
   - Upload time for a file, from request to successful, reliable storage
   - Download time for a file, from request to download completion

   Measure each file size at least 100 times and report the measured times with their average, median, min and max values.
7. Calculate and report the gains in download/upload time and compare to your analytical results.

Task 3: Delta Encoding As Compression

Design & Development

1. Expand the system from Task 1 to support delta compression based on VCDIFF (e.g., xdelta in Linux/MACOS). For example, after the file is received in the system, the lead node identifies whether it is a new file or a new version of an existing file. Then it proceeds to perform a delta encoding of the file using *either (a) the first version or (b) all previous versions as previous knowledge to the delta encoder*. Thus, the lead node will need to access a copy of the relevant previous version(s) to compute the deltas and then placing copies of the new delta in 3 different nodes selected. The goal is to reduce the footprint of each uploaded file exploiting its previous versions.

Analysis

2. Calculate the upload time, expanding the analysis from Task 1 in order to introduce the effect of processing (delta encoding) during the upload time. Consider that compression can be carried out at R_{enc} bps and that the resulting size of the delta for the i-th version is $B_{delta,i}$ bytes. Keep also in mind that you will need to access the first version ($B_{delta,1}$) or all previous versions ($B_{delta,1}$, $B_{delta,2}$, …, $B_{delta,i-1}$) to compute the delta.

3. Calculate the total download time of a file's i-th version when the decompression speed is R_{dec} bps. Download starts when the client sends the request, and ends when the last byte of the file has finished transferring. Consider the same connection bandwidth as in Point 2 above. Consider that downloading also requires to bring either the delta of the first version or all deltas (from 1 to i) to reconstruct the i-th version of the file.

4. Compare the upload and download time analysis to the results in Task 1. More specifically, compute the potential gain by dividing the upload time for Task 1 by the upload time for Task 2. Repeat the same process for download times. Report your computed gains based on the various system variables.

Measurements

5. Measure the size of the deltas of each file prior to upload. Measure the compression/decompression times. This will help you use the analysis to predict the gains with respect to the previous section. Report on the predicted gains and compare to those in Task 2.
6. Measure for cases $N = 6, 12, 24$ focusing on upload time (including the time to generate all replicas and store them correctly) as well as the total download time. Like before, use the files in the dataset directly for these tests as a baseline for performance. Describe and explain the results.
   Metrics:

- Upload time for a file, from request to successful, reliable storage
- Download time for a file, from request to download completion using option (a) for delta representations
- Download time for a file, from request to download completion using option (b) for delta representations

Measure the upload/download process at least 100 times and report the measured times with their average, median, min and max values.

7. Calculate and report the gains in download/upload time and compare to your analytical results. You need to report for both cases: first version serving as prior information and all previous versions serving as prior information for the delta. Compare against the gains of Task 2. Discuss your results overall.